

Universidad Nacional del Sur
Departamento de Ciencias e Ingeniería de la Computación
Ingeniería en Sistemas de Información

Guía Integral de Estudio y Defensa

ManyHands: Orquestación de Agentes de Lenguaje mediante Contratos, Grafos Tipados y Verificación por Evidencia

Material de Preparación para la Reunión de Tesis con Directores

Tesista: Francisco Clarke (LU: 120547)

Directora: Dra. Ana Gabriela Maguitman

Codirector: Dr. Federico Martín Schmidt

Fecha: Agosto de 2026

Propósito de este documento: Servir como texto de estudio integral, estructurado y de lectura fluida para dominar todos los conceptos, decisiones arquitectónicas, estrategias de implementación y justificaciones metodológicas del proyecto ManyHands, facilitando una defensa oral clara, rigurosa y convincente frente a los directores de tesis.

Índice

1. El «Pitch» y la Visión General: ¿Qué es ManyHands?	2
1.1. El problema fundamental del desarrollo con agentes de IA	2
1.2. La propuesta de ManyHands: Ingeniería guiada por contratos	2
2. El Viaje de una Tarea: Ciclo de Vida de Extremo a Extremo	3
3. Los Pilares Arquitectónicos Fundamentales (80 % del Sistema)	4
3.1. 1. El Grafo Híbrido y las 4 Relaciones Canónicas Tipadas	4
3.1.1. Las 4 Relaciones Tipadas (¿Por qué no una arista de dependencia genérica?)	4
3.2. 2. El Sistema de Contratos Inmutables	4
3.3. 3. Seguridad y Aislamiento Físico en Workers	5
3.4. 4. La Matriz de Evidencias (Evidence Matrix)	5
3.5. 5. Event Sourcing y Daemon Local	6
4. La Política de Granularidad Adaptativa (20 % del Sistema)	6
4.1. La Paradoja de la Granularidad	6
4.2. Por qué falló el scoring continuo (La historia de los 83 casos)	7
4.3. La solución: Política de Granularidad Categórica 4.0	7
5. Preguntas Típicas de los Profesores y Cómo Responderlas	8
6. Glosario Rápido y Conceptos Clave (<i>Cheat Sheet</i>)	10

1. El «Pitch» y la Visión General: ¿Qué es ManyHands?

1.1. El problema fundamental del desarrollo con agentes de IA

En los últimos años, los Modelos de Lenguaje de Gran Escala (LLMs) han demostrado una impresionante capacidad para escribir y editar código. Herramientas como Codex o Claude Code permiten a un modelo leer un archivo, ejecutar comandos en terminal y corregir errores sintácticos de forma interactiva.

Sin embargo, cuando intentamos utilizar estos agentes en **proyectos de software reales y de escala intermedia a grande**, surgen dos grandes fallas en los enfoques actuales:

1. **El enfoque de Agente Único (Monolítico):** Si le entregamos un requerimiento complejo que toca varias capas (ej. base de datos, lógica de negocio y endpoints de API) a un solo agente, el modelo sufre de *saturación de contexto* (se marea con demasiados archivos en su ventana de atención), empieza a inventar clases inexistentes (**alucinación**), comete *violaciones de alcance* (modifica archivos que nadie le pidió tocar) o rompe código no relacionado.
2. **El enfoque Multi-Agente Conversacional tradicional (ChatDev, MetaGPT):** Para solucionar lo anterior, la literatura propuso poner a varios agentes a «charlar» entre sí en lenguaje natural simulando una empresa de software (un agente hace de Product Manager, otro de Programador, otro de Tester). **En la práctica, esto falla en repositorios reales**: hablar en lenguaje natural acumula ambigüedades, los agentes no respetan las interfaces pactadas, no hay control de versiones riguroso por tarea y, al momento de juntar el código, se producen conflictos destructivos de integración.

1.2. La propuesta de ManyHands: Ingeniería guiada por contratos

ManyHands no es un chatbot ni un sistema de discusión libre entre modelos. Es una **plataforma local de orquestación de software**.

La Tesis Central de ManyHands en una frase

ManyHands sustituye la conversación libre entre agentes por un plano formal de **contratos inmutables**, un **grafo dirigido acíclico híbrido con relaciones tipadas**, **aislamiento físico estricto en Git worktrees** y una **disciplina de verificación basada en evidencia física sobre commits exactos**.

La analogía de la construcción: En una obra de ingeniería civil, el plomero, el electricista y el albañil no se coordinan teniendo charlas informales libres todos los días. Trabajan sobre **planos arquitectónicos con contratos de interfaz claros** (por dónde pasa cada caño, qué voltaje soporta la pared) y cada uno opera en su sector sin interferir físicamente con el otro. Cuando terminan, un inspector independiente prueba la presión del agua y la corriente eléctrica antes de dar la habilitación final. ManyHands aplica exactamente este rigor de ingeniería civil al desarrollo de software con agentes de IA.

2. El Viaje de una Tarea: Ciclo de Vida de Extremo a Extremo

Para explicarle a los profesores cómo funciona el sistema, lo más efectivo es describir el camino paso a paso que recorre un requerimiento desde que el usuario lo ingresa hasta que se entrega el código en Git.

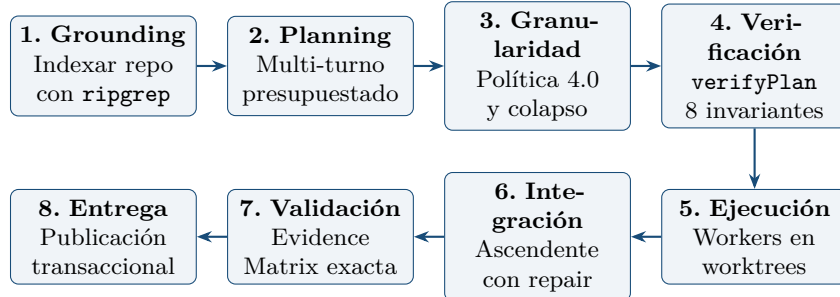


Figura 1: Flujo secuencial de ejecución de un requerimiento en ManyHands.

1. **Fase 1: Grounding del repositorio (`repository-index`):** Antes de planificar, el sistema toma una fotografía exacta del repositorio anclada al commit de Git actual (`RepositorySnapshot`). Utiliza `ripgrep` para descubrir archivos respetando `.gitignore` y construye un modelo estructurado (`RepositoryModel`) con los módulos, paquetes y scripts de prueba existentes.
2. **Fase 2: Planificación Progresiva Presupuestada (`PlanningEngine`):** El usuario ingresa su requerimiento (ej. «agregar soporte de prioridades a los pedidos»). El motor de planificación interactúa con el LLM pero **bajo un presupuesto estricto** (límite de llamadas, consultas y bytes). El LLM no recibe todo el código del tirón, sino que hace consultas puntuales al índice del repositorio. Además, el motor calcula un hash de estado causal para detectar si el modelo cae en un bucle infinito de reparación y abortar a tiempo (*Causal Loop Prevention*).
3. **Fase 3: Evaluación de Granularidad (`GranularityPolicy`):** El plan propuesto pasa por la Política de Granularidad 4.0. Si el corte propuesto es innecesario (no aporta paralelismo ni pruebas aisladas y cabe en un solo intento), el sistema **colapsa automáticamente las tareas en una sola hoja cohesiva** sin perder criterios de prueba.
4. **Fase 4: Verificación Estática de Invariantes (`verifyPlan`):** Antes de tocar una sola línea de código, un verificador determinista somete el plan a **8 categorías de invariantes matemáticos** (que no haya dependencias circulares, que ningún archivo sea editado por dos agentes a la vez sin orden previo, que todas las rutas sean seguras, etc.). Si pasa, se compila a un grafo ejecutable inmutable (`GraphRevision`).
5. **Fase 5: Ejecución Concurrente Aislada en Workers (`execution-core`):** El scheduler calcula qué tareas están listas para comenzar y las despacha en paralelo. Cada agente corre en su propio **Git worktree independiente** (una carpeta aislada con su propia copia de trabajo). El sistema aplica un *Scope Guard* estricto: si un agente intenta

modificar un archivo que no tenía autorizado en su contrato, su intento es abortado y rechazado de inmediato.

6. **Fase 6: Integración Ascendente y Reparación Semántica:** A medida que las hojas terminan, los nodos padres (compuestos) toman los parches de sus hijos en un worktree limpio y los integran de abajo hacia arriba (*bottom-up*), ejecutando pruebas de integración sobre las interfaces compartidas (*seams*). Si hay un conflicto, un agente especializado intenta una reparación semántica acotada.
7. **Fase 7: Matriz de Evidencias y Verificación Exacta:** No alcanza con que el agente diga «terminé». El sistema corre las pruebas sobre el commit exacto, verifica que las pruebas pasadas no hayan sido alteradas o debilitadas (*Test Integrity Guard*) y exige ****controles negativos**** (demostrar que la prueba nueva fallaba en el código viejo).
8. **Fase 8: Publicación Transaccional (TransactionalDeliveryPublisher):** El orquestador genera un manifiesto final inmutable y aplica un avance rápido (*fast-forward*) en la rama de Git del usuario emitiendo un recibo criptográfico de entrega.

—

3. Los Pilares Arquitectónicos Fundamentales (80 % del Sistema)

Esta sección detalla los componentes clave de la arquitectura para que puedas explicar su diseño conceptual y responder cualquier pregunta sobre cómo están contruidos.

3.1. 1. El Grafo Híbrido y las 4 Relaciones Canónicas Tipadas

En ManyHands, los nodos del grafo tienen roles jerárquicos claros:

- **Nodo Raíz (*Goal Root*):** Representa el objetivo final y la entrega al usuario.
- **Nodos Compuestos (*Composite Nodes*):** Representan fronteras de integración (ej. el módulo de pedidos, la capa de API). Su trabajo no es escribir código, sino integrar y validar los resultados de sus hijos.
- **Nodos Hoja (*Leaf Nodes*):** Son las tareas atómicas asignadas a un agente en un worktree.

3.1.1. Las 4 Relaciones Tipadas (¿Por qué no una arista de dependencia genérica?)

En los grafos tradicionales, solo existe una flecha genérica «A depende de B». En ManyHands demostramos que en ingeniería de software mezclar distintos tipos de dependencia genera problemas graves. Por eso definimos 4 relaciones canónicas:

3.2. 2. El Sistema de Contratos Inmutables

En lugar de pasarle al agente un prompt largo y desestructurado, cada tarea recibe un ****paquete hermético de contratos**** (`TaskContractBundle`):

Cuadro 1: Las 4 relaciones canónicas del grafo de ManyHands y su significado.

Relación Tipada	¿Qué significa en la práctica?	Efecto en el Scheduler de ejecución
Jerarquía (parentId)	Pertenencia arquitectónica.	El nodo padre espera a que sus hijos terminen para integrarlos.
ArtifactRequirement	Un nodo necesita el archivo físico que produce otro.	**Orden estricto** : El consumidor se bloquea hasta que el artefacto exista.
SeamBinding	Dos nodos acuerdan una interfaz común (ej. una firma de función).	**Paralelismo** : Ambos pueden programar al mismo tiempo contra el contrato.
ResourceClaim	Dos nodos necesitan editar el mismo archivo.	**Exclusión mutua** : Se ejecutan uno después del otro para no pisarse.

- **ScopeContract (Contrato de Alcance)**: Define la lista blanca cerrada de archivos que el agente tiene permitido leer y modificar (`writePaths`). Si el agente toca un archivo fuera de esa lista, el orquestador rechaza el cambio automáticamente.
- **SeamContract (Contrato de Costura / Interfaz)**: Define las interfaces públicas acordadas antes de escribir código (tipos exportados, parámetros, nombres de endpoints).
- **ArtifactContract (Contrato de Artefacto)**: Define qué archivo o entregable exacto debe producir el nodo.
- **ValidationObligation (Obligación de Validación)**: Define qué comando de test debe ejecutarse para considerar que la tarea es correcta y qué nivel de severidad tiene.

3.3. 3. Seguridad y Aislamiento Físico en Workers

¿Cómo garantizamos que ejecutar agentes no rompa la máquina del desarrollador?

1. **Pool de Git Worktrees**: Cada agente trabaja en una carpeta aislada gestionada por Git. Si el agente borra archivos o rompe dependencias, el directorio principal del usuario queda 100 % intacto. Al finalizar, el worktree se limpia con `git reset -hard` y `git clean -fdx`.
2. **Scope Guard (Deny-First)**: Se audita el diff real de Git. Se bloquean rutas con secuencias de escape (`../`) y enlaces simbólicos engañosos.
3. **Supervisión del árbol de procesos del SO**: Si un agente se cuelga o agota su tiempo, ManyHands utiliza `taskkill /t /f` en Windows o `SIGKILL` en Unix para matar recursivamente a todos los procesos hijos sin dejar servidores zombies consumiendo memoria.
4. **Saneamiento de entorno**: Los agentes no heredan las variables de entorno de la máquina. Se eliminan claves de API y credenciales para evitar fugas de información.

3.4. 4. La Matriz de Evidencias (Evidence Matrix)

ManyHands no confía en la palabra del modelo ni en una salida de texto de la consola. La aprobación se basa en evidencia formal:

- **Validación sobre el commit exacto:** Los tests se corren sobre el hash SHA exacto generado en Git.
- **Controles Negativos Obligatorios:** Si el agente crea un nuevo test para validar su código, el orquestador corre ese test contra el código viejo antes de su cambio. ****El test debe fallar en el código viejo****. Si el test pasaba en el código viejo, significa que el test es trivial o no evalúa nada real (*falso positivo*).
- **Test Integrity Guard (Anti-trampas):** Si el agente intenta hacer «trampa» borrando aserciones de tests existentes o agregando directivas para ignorar pruebas (`it.skip`, `xit`), el verificador de AST detecta la maniobra y clasifica el intento como `fraudulent_evidence`, descartándolo.
- **Flaky Policy (Anti-intermitencia):** Si una prueba pasa y falla de forma aleatoria sin cambios de código, se marca como inestable y bloquea la entrega.

3.5. 5. Event Sourcing y Daemon Local

Toda la historia del sistema se guarda en un ****journal de solo anexión (*append-only log*) de eventos de dominio**** en formato JSONL:

- ****Tolerancia a fallos:**** Si la máquina se apaga o se corta la luz, al reiniciar el sistema lee los eventos del disco y reconstruye en memoria exactamente el mismo estado sin pérdida de datos.
- ****Desacoplamiento de la UI:**** Un daemon en segundo plano custodia la ejecución real. La interfaz web es solo una vista cliente que puede abrirse o cerrarse sin interrumpir el trabajo de los agentes.

—

4. La Política de Granularidad Adaptativa (20 % del Sistema)

Esta es una sección clave para tu defensa. La granularidad responde a la pregunta metodológica: ****¿Cómo decide el sistema cuándo dividir una tarea y cuándo dejarla como una sola unidad?****

4.1. La Paradoja de la Granularidad

- Si ****no divides**** una tarea compleja → El agente sufre saturación de contexto y falla.
- Si ****divides de más**** (sobre-fragmentación) → Gastas miles de tokens coordinando interfaces, y el costo de integrar y resolver conflictos supera el costo de haber hecho la tarea junta.

4.2. Por qué falló el scoring continuo (La historia de los 83 casos)

En las primeras versiones del proyecto se intentó usar una fórmula matemática tradicional: se calculaba un puntaje continuo ponderando 4 variables (radio de alcance, impacto de interfaz, superficie de validación y masa de código) y se comparaba contra un umbral $\theta = 3,5$.

Al probar esa fórmula contra **83 casos históricos de cortes reales**, se descubrió algo contundente:

1. El número no tenía unidades físicas (no medía tokens ni minutos).
2. Al mover el umbral en todo el rango $[0, 0,20]$, el sistema tomaba exactamente las mismas decisiones. El umbral era un adorno matemático que no se podía calibrar ni defender.
3. En 83 casos, la fórmula solo aportó 10 colapsos frente a la regla simple de dividir siempre, y ningún factor explicaba por qué.

4.3. La solución: Política de Granularidad Categórica 4.0

Eliminamos los números arbitrarios y los reemplazamos por **3 límites físicos** y **3 razones categóricas expresables en lenguaje natural**:

Los 3 Límites Físicos de una Hoja

Una tarea solo puede ser resuelta por un único agente si cumple los tres límites:

1. Tokens de contexto $\leq 24\,000$ tokens.
2. Archivos existentes leídos/modificados ≤ 40 archivos.
3. Archivos nuevos a crear ≤ 12 archivos. (Lección del caso Warehouse W2: crear muchos archivos desde cero satura al agente aunque el repo sea chico).

Las 3 Razones Categóricas para Dividir (SPLIT)

Una tarea solo se divide si se cumple al menos una de estas tres condiciones:

1. **doesNotFit (No cabe)**: Supera alguno de los 3 límites físicos anteriores. Dividir es la única forma de que sea ejecutable.
2. **runsInParallel (Paralelismo real)**: Al analizar el grafo de dependencias de artefactos, al menos dos sub-tareas pueden arrancar al mismo tiempo. (Nota: una cadena estrictamente secuencial $A \rightarrow B \rightarrow C$ no compra tiempo de reloj y recibe `runsInParallel = false`).
3. **verifiableApart (Verificable por separado)**: Cada sub-tarea es dueña de una prueba que ningún hermano comparte. Si una sub-tarea falla, la evidencia de éxito de las otras no se pierde.

La Regla de Decisión y Colapso: Si una partición propuesta **no cumple ninguna de las 3 razones**, el sistema ejecuta `applyGranularitySelection` y **colapsa determinísticamente** la

división en una sola hoja cohesiva**, absorbiendo todos los criterios y eliminando las interfaces intermedias innecesarias.

El Banco de Regresión Offline: Tenemos una suite de pruebas con 83 casos congelados (`granularity-bank-baseline.json`). Cada vez que se toca una línea de la política, se corre la suite determinista para comprobar que las decisiones se mantengan estables y auditables.

5. Preguntas Típicas de los Profesores y Cómo Responderlas

Esta sección contiene las preguntas conceptuales y de diseño que los profesores y directores de tesis suelen formular, junto con la respuesta recomendada y fundamentada.

Pregunta de los profesores: 1. *¿Por qué creaste ManyHands en lugar de usar frameworks existentes como LangChain, AutoGen o MetaGPT?*

Respuesta clave: «Los frameworks como AutoGen o MetaGPT están basados en el paradigma conversacional: simulan una reunión entre personas donde los agentes intercambian mensajes de chat en lenguaje natural. Eso funciona bien para demos o para generar scripts pequeños desde cero (*greenfield*), pero fracasa en repositorios reales con miles de líneas de código porque el lenguaje natural acumula ambigüedades, no hay control de versiones por intento y los agentes terminan desalineando sus interfaces.

»ManyHands aborda el problema desde la ****ingeniería de software****: en lugar de chat libre, implementamos un plano de control con contratos formales inmutables, aislamiento de procesos en Git worktrees, verificación estática antes de ejecutar y validación de pruebas sobre el commit físico exacto. Es la diferencia entre una charla informal y un plano de ingeniería.»

Pregunta de los profesores: 2. *¿Cómo evitas que dos agentes entren en conflicto si necesitan editar el mismo archivo?*

Respuesta clave: «Lo resolvemos en dos niveles complementarios:

»1. ****En la planificación (Prevención):**** Nuestro modelo de grafo incluye la relación tipada `ResourceClaim`. Si dos tareas declaran que necesitan modificar el mismo archivo, el scheduler detecta el conflicto y las serializa automáticamente (las pone en olas distintas en lugar de correrlas en paralelo). 2. ****En la integración (Resolución):**** Cuando los nodos hijos terminan, el nodo padre ejecuta la integración en un worktree limpio. Si surge un conflicto semántico, el orquestador dispara un intento de reparación automática con el contexto global del padre. Si no converge, escala una decisión estructurada al usuario en vez de introducir código roto.»

Pregunta de los profesores: 3. *¿Cómo aseguras que un agente no genere pruebas falsas o triviales que siempre den verde para «aprobar»?*

Respuesta clave: «Implementamos la **Matriz de Evidencias** con dos mecanismos de seguridad fundamentales:

»1. ****Controles Negativos Obligatorios:**** Cuando un agente introduce un nuevo test, el orquestador ejecuta esa prueba sobre el commit base anterior a su cambio. La prueba está obligada a fallar en el código previo. Si pasa en el código viejo, se descarta por considerarse un falso positivo no sensible. 2. ****Test Integrity Guard:**** Analizamos el AST del código de prueba para detectar si el agente intentó borrar aserciones previas, debilitar comparaciones o usar directivas para saltar pruebas (`it.skip`). Si se detecta manipulación, el intento se marca como evidencia fraudulenta y se rechaza.»

Pregunta de los profesores: 4. *¿Por qué la política de granularidad no usa Machine Learning o una optimización matemática?*

Respuesta clave: «Inicialmente probamos un enfoque de scoring continuo ponderando variables contra un umbral. Al evaluarlo empíricamente sobre 83 casos históricos reales, descubrimos que los puntajes continuos eran adimensionales, no tenían unidades físicas y el umbral numérico degeneraba (daba exactamente igual en cualquier valor entre 0 y 0.20).

»Por eso adoptamos la ****Política Categórica 4.0****: la granularidad se decide por propiedades físicas y estructurales discretas (`doesNotFit`, `runsInParallel`, `verifiableApart`). Esto elimina los hiperparámetros libres y hace que cada decisión de partición o colapso tenga una justificación explicable en lenguaje natural y sea 100 % reproducible mediante un banco de regresión offline.»

Pregunta de los profesores: 5. *¿Qué pasa si se corta la luz, se reinicia la máquina o se cae el proceso del daemon en medio de una ejecución?*

Respuesta clave: «El sistema utiliza una arquitectura basada en ****Event Sourcing**** con un journal append-only en formato JSONL. Ningún estado mutable vive solo en memoria. Cada transición de estado, inicio de intento o resultado de prueba se escribe a disco de forma atómica. »Al reiniciar el daemon, el sistema lee la secuencia de eventos desde el principio, ejecuta la función reductora pura y reconstruye el estado exacto del grafo y de los workers en memoria. Además, las concesiones de ejecución utilizan ***testigos durables*** (***witness leases***), por lo que cualquier proceso zombie que haya quedado colgado es detectado y descartado sin corromper el repositorio.»

Pregunta de los profesores: 6. ¿Cuál es la autoría de lo que hiciste frente a herramientas externas?

Respuesta clave: «Las herramientas externas que usamos son los ejecutores de lenguaje (Codex CLI o Claude Code) como motores de inferencia de bajo nivel, Git para el almacenamiento de versiones y `ripgrep` para la búsqueda rápida de texto.

»**Toda la arquitectura de orquestación es desarrollo propio**: el diseño e implementación del modelo de grafo híbrido y sus 4 relaciones tipadas, el motor de contratos formales con esquemas Zod, el motor de planificación presupuestado con prevención de bucles causales, el verificador de 8 invariantes de plan, el pool de worktrees con scope guard en tiempo de ejecución, el motor de integración ascendente, la matriz de evidencias con controles negativos, el publicador transaccional de entregas, la política de granularidad categórica y la interfaz de supervisión accesible.»

Pregunta de los profesores: 7. ¿Qué es exactamente una «Costura» o Seam en la arquitectura?

Respuesta clave: «El concepto de *Seam* proviene de la literatura de diseño de software (Michael Feathers y David Parnas) y representa una frontera de interfaz compartida entre dos módulos. En ManyHands, un `SeamContract` formaliza esa frontera antes de que los agentes empiecen a programar (por ejemplo, define el nombre de una función, sus tipos de entrada y salida).

»Esto tiene un impacto enorme en la concurrencia: al congelar la costura de antemano mediante un contrato, el agente que implementa la función y el agente que la consume pueden trabajar **en paralelo al mismo tiempo**, porque ambos programan contra una especificación formal y no necesitan esperar a que el otro termine.»

6. Glosario Rápido y Conceptos Clave (*Cheat Sheet*)

Utiliza estos términos canónicos con precisión durante tu conversación con los directores:

Run	La unidad atómica de producto en ManyHands: transforma un objetivo en lenguaje natural en un commit verificado y entregado en Git.
GraphRevision	Revisión inmutable del grafo de tareas que contiene nodos, relaciones y paquetes de contratos.
Composite Node	Nodo compuesto que representa una frontera arquitectónica y es responsable de integrar y validar los resultados de sus hijos.
Leaf Node	Nodo hoja que representa una tarea atómica de código ejecutada por un agente en un worktree aislado.
SeamContract	Contrato de interfaz pública compartida pactado a priori para permitir desarrollo concurrente.

ScopeContract	Lista blanca cerrada de rutas de archivos que un agente tiene autorización de modificar (<code>writePaths</code>).
Scope Guard	Mecanismo de contención en tiempo de ejecución que rechaza cualquier cambio fuera del alcance permitido.
PlanningBudget	Presupuesto multidimensional inmutable que limita llamadas LLM, consultas al repositorio y bytes leídos.
CausalDigest	Hash criptográfico que previene bucles infinitos de reparación (<code>*no__progress*</code>) en la planificación.
verifyPlan	Verificador estático determinista que audita 8 categorías de invariantes antes de compilar el grafo.
EvidenceMatrix	Matriz que vincula criterios de aceptación con ejecuciones de pruebas sobre el commit exacto, auditando controles negativos.
Negative Control	Validación obligatoria de que una prueba nueva falle cuando se corre contra el código previo.
Test Integrity Guard	Verificador que detecta si un agente eliminó aserciones o saltó tests (<code>it.skip</code>) de forma fraudulenta.
GranularityPolicy 4.0	Política categórica que decide cortes o colapsos basada en capacidad física, paralelismo real y pruebas aisladas.
Event Sourcing	Patrón de persistencia donde la historia del run es un log inmutable append-only de eventos de dominio.